# *Embedded Bitwise Blueprint*

## *65 Challenges to Perfect Your Skills*

**1) Bitwise OR**
result = x | y;

**2) Bitwise NOT**
x = ~x;

**3) Bitwise AND**
result = x & y;

**4) Bitwise XOR**
result = x ^ y;

**Program for the above:**

```c
#include <stdio.h>
#include <stdlib.h>

char* decimalToBinary(int num) {
    char* binary = (char*)malloc(33);  // 32 bits + null terminator for an int
    binary[32] = '\0';
    int index = 31;

    for(int i = 0; i < 32; i++) {
        if((num & 1) == 1)
            binary[index] = '1';
        else
            binary[index] = '0';
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x, y;
    int result;

    printf("Enter two integers (x y): ");
    scanf("%d %d", &x, &y);

    // Bitwise OR
    result = x | y;
    printf("Bitwise OR of %d (%s) and %d (%s): %d (%s)\n", x, decimalToBinary(x), y, decimalToBinary(y), result, decimalToBinary(result));

    // Bitwise NOT
    x = ~x;
    printf("Bitwise NOT of x: %d (%s)\n", x, decimalToBinary(x));
    x= ~x;

    // Bitwise AND
    result = x & y;
    printf("Bitwise AND of %d (%s) and %d (%s): %d (%s)\n", x, decimalToBinary(x), y, decimalToBinary(y), result, decimalToBinary(result));

    // Bitwise XOR
    result = x ^ y;
    printf("Bitwise XOR of %d (%s) and %d (%s): %d (%s)\n", x, decimalToBinary(x), y, decimalToBinary(y), result, decimalToBinary(result));

    return 0;
}
```

**Output:**

```
~ ./basic_bitwise
Enter two integers (x y): 5 6
Bitwise OR of 5 (00000000000000000000000000000101) and 6 (00000000000000000000000000000110): 7 (00000000000000000000000000000111)
Bitwise NOT of x: -6 (11111111111111111111111111111010)
Bitwise AND of 5 (00000000000000000000000000000101) and 6 (00000000000000000000000000000110): 4 (00000000000000000000000000000100)
Bitwise XOR of 5 (00000000000000000000000000000101) and 6 (00000000000000000000000000000110): 3 (00000000000000000000000000000011)
```

**5) Left Shift**
x = x << n;

**6) Right Shift**
x = x >> n;

Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>

char* decimalToBinary(int num) {
    char* binary = (char*)malloc(33);  // 32 bits + null terminator for an int
    binary[32] = '\0';
    int index = 31;

    for(int i = 0; i < 32; i++) {
        if((num & 1) == 1)
            binary[index] = '1';
        else
            binary[index] = '0';
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x, n;

    printf("Enter an integer (x): ");
    scanf("%d", &x);
    printf("Enter number of positions for shifting (n): ");
    scanf("%d", &n);

    // Left Shift
    int leftShiftResult = x << n;
    printf("Left Shift of %d (%s) by %d positions: %d (%s)\n", x, decimalToBinary(x), n, leftShiftResult, decimalToBinary(leftShiftResult));

    // Right Shift
    int rightShiftResult = x >> n;
    printf("Right Shift of %d (%s) by %d positions: %d (%s)\n", x, decimalToBinary(x), n, rightShiftResult, decimalToBinary(rightShiftResult));

    return 0;
}
```

**Output:**

```
~ ./bit_shifting
Enter an integer (x): 9
Enter number of positions for shifting (n): 2
Left Shift of 9 (00000000000000000000000000001001) by 2 positions: 36 (00000000000000000000000000100100)
Right Shift of 9 (00000000000000000000000000001001) by 2 positions: 2 (00000000000000000000000000000010)
```

**7) Setting a Bit (Bitwise OR)**
x = x | (1 << n);

**8) Clearing a Bit (Bitwise AND)**
x = x & ~(1 << n);

**9) Toggling a Bit (Bitwise XOR)**
x = x ^ (1 << n);

**10) Checking a Bit**
bit = (x & (1 << n)) != 0;

## Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>

char* decimalToBinary(int num) {
    char* binary = (char*)malloc(33);  // 32 bits + null terminator for an int
    binary[32] = '\0';
    int index = 31;

    for(int i = 0; i < 32; i++) {
        if((num & 1) == 1)
            binary[index] = '1';
        else
            binary[index] = '0';
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x, n;
    int bit;

    printf("Enter an integer (x): ");
    scanf("%d", &x);
    printf("Enter the bit position (0-based index) to manipulate (n): ");
    scanf("%d", &n);

    printf("\nValue of x in binary:%s\n",decimalToBinary(x));

    // Setting a Bit (Bitwise OR)
    int setBitResult = x | (1 << n);
    printf("\nSetting bit %d of %d (%s): %d (%s)\n", n, x, decimalToBinary(x), setBitResult, decimalToBinary(setBitResult));

    // Clearing a Bit (Bitwise AND)
    int clearBitResult = x & ~(1 << n);
    printf("Clearing bit %d of %d (%s): %d (%s)\n", n, x, decimalToBinary(x), clearBitResult, decimalToBinary(clearBitResult));

    // Toggling a Bit (Bitwise XOR)
    int toggleBitResult = x ^ (1 << n);
    printf("Toggling bit %d of %d (%s): %d (%s)\n", n, x, decimalToBinary(x), toggleBitResult, decimalToBinary(toggleBitResult));

    // Checking a Bit
    bit = (x & (1 << n)) != 0;
    printf("Checking bit %d of %d (%s): %s\n", n, x, decimalToBinary(x), bit ? "Set" : "Not Set");

    return 0;
}
```

## Output:

```
~ ./set_clear_toggle_check
Enter an integer (x): 23
Enter the bit position (0-based index) to manipulate (n): 3

Value of x in binary:00000000000000000000000000010111

Setting bit 3 of 23 (00000000000000000000000000010111): 31 (00000000000000000000000000011111)
Clearing bit 3 of 23 (00000000000000000000000000010111): 23 (00000000000000000000000000010111)
Toggling bit 3 of 23 (00000000000000000000000000010111): 31 (00000000000000000000000000011111)
Checking bit 3 of 23 (00000000000000000000000000010111): Not Set
~
```

**11) Rotate Bits (Left)**

int rotate_left = (x << n) | (x >> (sizeof(x)*8 - n));

**12) Rotate Bits (Right)**

int rotate_right = (x >> n) | (x << (sizeof(x)*8 - n));

## Program for above:

```c
#include <stdio.h>
#include <stdlib.h>

char* decimalToBinary(int num) {
    char* binary = (char*)malloc(33);  // 32 bits + null terminator for an int
    binary[32] = '\0';
    int index = 31;

    for(int i = 0; i < 32; i++) {
        if((num & 1) == 1)
            binary[index] = '1';
        else
            binary[index] = '0';
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x, n;

    printf("Enter an integer (x): ");
    scanf("%d", &x);
    printf("Enter the number of positions to rotate (n): ");
    scanf("%d", &n);

    // Rotate Bits (Left)
    int rotate_left = (x << n) | (x >> (sizeof(x)*8 - n));
    printf("\nRotate %d (%s) to the left by %d positions: %d (%s)\n", x, decimalToBinary(x), n, rotate_left, decimalToBinary(rotate_left));

    // Rotate Bits (Right)
    int rotate_right = (x >> n) | (x << (sizeof(x)*8 - n));
    printf("Rotate %d (%s) to the right by %d positions: %d (%s)\n", x, decimalToBinary(x), n, rotate_right, decimalToBinary(rotate_right));

    return 0;
}

```

## Output:

```
~ ./rotate_bits
Enter an integer (x): 25
Enter the number of positions to rotate (n): 5

Rotate 25 (00000000000000000000000000011001) to the left by 5 positions: 800 (00000000000000000000001100100000)
Rotate 25 (00000000000000000000000000011001) to the right by 5 positions: -939524096 (11001000000000000000000000000000)
```

## 13) Isolate Rightmost 1-Bit

int rightmost_set_bit = x & (-x);

## 14) Clear Rightmost 1-Bit

x = x & (x-1);

## Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>

char* decimalToBinary(int num) {
    char* binary = (char*)malloc(33);  // 32 bits + null terminator for an int
    binary[32] = '\0';
    int index = 31;

    for(int i = 0; i < 32; i++) {
        if((num & 1) == 1)
            binary[index] = '1';
        else
            binary[index] = '0';
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x;

    printf("Enter an integer (x): ");
    scanf("%d", &x);

    // Isolate Rightmost 1-Bit
    int rightmost_set_bit = x & (-x);
    printf("\nIsolate rightmost 1-bit of %d (%s): %d (%s)\n", x, decimalToBinary(x), rightmost_set_bit, decimalToBinary(rightmost_set_bit));

    // Clear Rightmost 1-Bit
    x = x & (x-1);
    printf("Clear rightmost 1-bit: %d (%s)\n", x, decimalToBinary(x));

    return 0;
}

```

## Output:

```
~ ./right_most_bit
Enter an integer (x): 75

Isolate rightmost 1-bit of 75 (00000000000000000000000001001011): 1 (00000000000000000000000000000001)
Clear rightmost 1-bit: 74 (00000000000000000000000001001010)
```

## 15) Multiply by 2:

x = x << 1;

## 16) Divide by 2:

x = x >> 1;

## Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>

char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x;

    printf("Enter an integer (x): ");
    scanf("%d", &x);

    // Multiply by 2
    int multiplyBy2 = x << 1;
    printf("\nMultiplying %d (%s) by 2: %d (%s)\n", x, decimalToBinaryWithSpace(x), multiplyBy2, decimalToBinaryWithSpace(multiplyBy2));

    // Divide by 2
    int divideBy2 = x >> 1;
    printf("Dividing %d (%s) by 2: %d (%s)\n", x, decimalToBinaryWithSpace(x), divideBy2, decimalToBinaryWithSpace(divideBy2));

    return 0;
}

```

## Output:

```
~ ./by_2
Enter an integer (x): 379

Multiplying 379 ( 0000 0000 0000 0000 0001 0111 1011) by 2: 758 ( 0000 0000 0000 0000 0010 1111 0110)
Dividing 379 ( 0000 0000 0000 0000 0001 0111 1011) by 2: 189 ( 0000 0000 0000 0000 0000 1011 1101)
```

**17) Flip all bits:**
x = ~x;

**18) Compute 2^x:**
int result = 1 << x;

**19) Check if a number is a power of 2:**

bool isPowerOf2 = x && (!(x & (x - 1)));

**Program for the above:**

```c
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x;

    printf("Enter an integer (x): ");
    scanf("%d", &x);

    // Flip all bits
    int flipped = ~x;
    printf("\nFlipping all bits of %d (%s): %d (%s)\n", x, decimalToBinaryWithSpace(x), flipped, decimalToBinaryWithSpace(flipped));

    // Compute 2^x
    int powerResult = 1 << x;
    printf("2^%d: %d (%s)\n", x, powerResult, decimalToBinaryWithSpace(powerResult));

    // Check if a number is a power of 2
    bool isPowerOf2 = x && (!(x & (x - 1)));
    printf("%d (%s) is %sa power of 2\n", x, decimalToBinaryWithSpace(x), isPowerOf2 ? "" : "not ");

    return 0;
}
```

**Output:**

```
~ ./flip_and_power_2
Enter an integer (x): 261

Flipping all bits of 261 ( 0000 0000 0000 0000 0001 0000 0101): -262 ( 1111 1111 1111 1111 1110 1111 1010)
2^261: 32 ( 0000 0000 0000 0000 0000 0010 0000)
261 ( 0000 0000 0000 0000 0001 0000 0101) is not a power of 2
```

**20) Turn off all bits except the rightmost set bit:**
x = x & (-x);

**21) Turn on all bits to the right of rightmost set bit:**
x = x | (x-1);

**Program for the above:**

```c
#include <stdio.h>
#include <stdlib.h>

char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x;

    printf("Enter an integer (x): ");
    scanf("%d", &x);

    // Turn off all bits except the rightmost set bit
    int rightmostSetBit = x & (-x);
    printf("\nTurn off all bits of %d (%s) except the rightmost set bit: %d (%s)\n", x, decimalToBinaryWithSpace(x), rightmostSetBit, decimalToBinaryWithSpace(rightmostSetBit));

    // Turn on all bits to the right of rightmost set bit
    int rightOfRightmost = x | (x-1);
    printf("Turn on all bits of %d (%s) to the right of rightmost set bit: %d (%s)\n", x, decimalToBinaryWithSpace(x), rightOfRightmost, decimalToBinaryWithSpace(rightOfRightmost));

    return 0;
}
```

**Output:**

```
~ ./bit_turning
Enter an integer (x): 3613

Turn off all bits of 3613 ( 0000 0000 0000 0000 1110 0001 1101) except the rightmost set bit: 1 ( 0000 0000 0000 0000 0000 0000 0001)
Turn on all bits of 3613 ( 0000 0000 0000 0000 1110 0001 1101) to the right of rightmost set bit: 3613 ( 0000 0000 0000 0000 1110 0001 1101)
~ vi bit_turning.c
```

**22) Bit Masking**
int masked_value = x & mask;

**23) Setting Multiple Bits:**

#define MASK 0x0F
x |= MASK;  // Set lower 4 bits

**24) Clearing Multiple Bits:**

#define MASK 0xF0
x &= ~MASK;  // Clear upper 4 bits

**Program for the above:**

```c
#include <stdio.h>
#include <stdlib.h>

// Function to convert decimal to binary with spaces for every 4 bits
char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x, mask;

    // 1) Bit Masking
    printf("Enter an integer (x): ");
    scanf("%d", &x);
    printf("Enter a mask: ");
    scanf("%d", &mask);

    int masked_value = x & mask;
    printf("\nBit Masking: %d (%s) AND %d (%s) = %d (%s)\n", x, decimalToBinaryWithSpace(x), mask, decimalToBinaryWithSpace(mask), masked_value, decimalToBinaryWithSpace(masked_value));

    // 2) Setting Multiple Bits
    #define MASK_SET 0x0F
    x |= MASK_SET;
    printf("\nSetting Multiple Bits: %d (%s)\n", x, decimalToBinaryWithSpace(x));

    // 3) Clearing Multiple Bits
    #define MASK_CLEAR 0xF0
    x &= ~MASK_CLEAR;
    printf("Clearing Multiple Bits: %d (%s)\n", x, decimalToBinaryWithSpace(x));

    return 0;
}
```

**Output:**

```
~ ./masking_set_multiple
Enter an integer (x): 337
Enter a mask: 3

Bit Masking: 337 ( 0000 0000 0000 0000 0001 0101 0001) AND 3 ( 0000 0000 0000 0000 0000 0000 0011) = 1 ( 0000 0000 0000 0000 0000 0000 0001)

Setting Multiple Bits: 351 ( 0000 0000 0000 0000 0001 0101 1111)
Clearing Multiple Bits: 271 ( 0000 0000 0000 0000 0001 0000 1111)
```

**25) Selective Bit Inversion:**
Invert specific bits (useful in scenarios like manipulating control registers):

x ^= MASK;

**26) Compute the Sign of an Integer:**

int sign = (x >> 31) | !!x;

**27) Conditional Negation:**

int r = (x >> 31 & (y ^ z)) ^ z;

## Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>

// Function to convert decimal to binary with spaces for every 4 bits
char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x, y, z, MASK;

    // 4) Selective Bit Inversion
    printf("Enter an integer (x): ");
    scanf("%d", &x);
    printf("Enter a mask for inversion: ");
    scanf("%d", &MASK);

    x ^= MASK;
    printf("\nSelective Bit Inversion: %d (%s)\n", x, decimalToBinaryWithSpace(x));

    // 5) Compute the Sign of an Integer
    int sign = (x >> 31) | !!x; // !!x converts x to 0 (for x=0) or 1 (for x!=0)
    printf("\nSign of %d: %d\n", x, sign);

    // 6) Conditional Negation
    printf("Enter two more integers (y and z): ");
    scanf("%d %d", &y, &z);

    int r = (x >> 31 & (y ^ z)) ^ z;
    printf("\nConditional Negation Result: %d (%s)\n", r, decimalToBinaryWithSpace(r));

    return 0;
}
```

## Output:

```
~ ./inversion_computeSign_negation
Enter an integer (x): 552
Enter a mask for inversion: 12

Selective Bit Inversion: 548 ( 0000 0000 0000 0000 0010 0010 0100)

Sign of 548: 1
Enter two more integers (y and z): 2336 2251

Conditional Negation Result: 2251 ( 0000 0000 0000 0000 1000 1100 1011)
```

**28) Determine if two integers have the same sign:**

bool sameSign = (x ^ y) >= 0;

**29) Isolate the rightmost bit that is set to 1:**

int rightmostBit = x & (-x);

**Program for the above:**

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h> // For bool data type

// Function to convert decimal to binary with spaces for every 4 bits
char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x, y;

    // 7) Determine if two integers have the same sign
    printf("Enter two integers (x and y): ");
    scanf("%d %d", &x, &y);

    bool sameSign = (x ^ y) >= 0;
    printf("\nDo %d and %d have the same sign? %s\n", x, y, sameSign ? "Yes" : "No");

    // 8) Isolate the rightmost bit that is set to 1
    printf("\nEnter an integer (x) to isolate its rightmost set bit: ");
    scanf("%d", &x);

    int rightmostBit = x & (-x);
    printf("\nRightmost set bit of %d: %d (%s)\n", x, rightmostBit, decimalToBinaryWithSpace(rightmostBit));

    return 0;
}
```

**Output:**

```
./sameSign_IsolateRightMostSeto1
Enter two integers (x and y): 25 23

Do 25 and 23 have the same sign? Yes

Enter an integer (x) to isolate its rightmost set bit: 2263

Rightmost set bit of 2263: 1 ( 0000 0000 0000 0000 0000 0000 0001)
```

## 30) Merge bits from two values according to a mask:

int result = (x & mask) | (y & ~mask);

## Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>

// Function to convert decimal to binary with spaces for every 4 bits
char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x, y, mask;

    // Input the values
    printf("Enter the first integer (x): ");
    scanf("%d", &x);
    printf("Enter the second integer (y): ");
    scanf("%d", &y);
    printf("Enter a mask: ");
    scanf("%d", &mask);

    // 9) Merge bits from two values according to a mask
    // The operation (x & mask) retains the bits in x where the mask has 1s.
    // The operation (y & ~mask) retains the bits in y where the mask has 0s.
    // The final OR operation combines the retained bits from x and y to produce the result.
    int result = (x & mask) | (y & ~mask);
    printf("\nMerged result of %d (%s) and %d (%s) with mask %d (%s) is: %d (%s)\n", x, decimalToBinaryWithSpace(x), y, decimalToBinaryWithSpace(y), mask, decimalToBinaryWithSpace(mask), result, decimalToBinaryWithSpace(result));

    return 0;
}
```

## Output:

```
~ ./merge_bits_2values
Enter the first integer (x): 6691
Enter the second integer (y): 3302
Enter a mask: 2

Merged result of 6691 ( 0000 0000 0000 0001 1010 0010 0011) and 3302 ( 0000 0000 0000 0000 1100 1110 0110) with mask 2 ( 0000 0000 0000 0000 0000 0000 0010) is: 3302 ( 0000 0000 0000 0000 1100 1110 0110)
```

## 31) Detect if a number has consecutive bits set:

bool hasConsecutiveOnes = (x & (x << 1)) != 0;

## Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h> // For bool data type

// Function to convert decimal to binary with spaces for every 4 bits
char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x;

    // Input the value
    printf("Enter an integer (x): ");
    scanf("%d", &x);

    // By shifting x to the left by 1, we move all bits one position to the left.
    // If x has two consecutive bits set, then x & (x << 1) will have at least one bit set.
    // For example, consider x = ...011... (with 2 consecutive set bits). After left shifting, it becomes ...110...
    // When we perform bitwise AND between x and (x << 1), it results in ...010... which is not zero.
    bool hasConsecutiveOnes = (x & (x << 1)) != 0;
    printf("\nDoes %d (%s) have consecutive set bits? %s\n", x, decimalToBinaryWithSpace(x), hasConsecutiveOnes ? "Yes" : "No");

    return 0;
}
```

## Output:

```
~ ./detect_consecutive_bits
Enter an integer (x): 7841

Does 7841 ( 0000 0000 0000 0001 1110 1010 0001) have consecutive set bits? Yes
```

## 32) Count Set Bits

```
count = 0;
while(x) {
   count += x & 1;
   x >>= 1;
}
```

## Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>

// Function to convert decimal to binary with spaces for every 4 bits
char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x;

    // Input the value
    printf("Enter an integer (x): ");
    scanf("%d", &x);

    // This algorithm works by repeatedly checking the least significant bit of x and then right shifting x.
    // The operation (x & 1) checks if the least significant bit of x is set.
    // If it's set, it adds 1 to the count.
    // The operation (x >>= 1) right shifts x, effectively dropping its least significant bit.
    // This process continues until x becomes zero.
    int count = 0;
    int originalX = x;  // Store the original value of x for later use
    while(x) {
        count += x & 1;
        x >>= 1;
    }
    printf("\nNumber of set bits in %d (%s) is: %d\n", originalX, decimalToBinaryWithSpace(originalX), count);

    return 0;
}
```

## Output:

```
~ ./count_set_bits
Enter an integer (x): 295841

Number of set bits in 295841 ( 0000 0000 0100 1000 0011 1010 0001) is: 7
```

## 33) Swap Bits

```
if (((x >> p) & 1) != ((x >> q) & 1)) {
    x ^= (1 << p) | (1 << q);
}
```

## Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>

// Function to convert decimal to binary with spaces for every 4 bits
char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x, p, q;

    // Input the values
    printf("Enter an integer (x): ");
    scanf("%d", &x);
    printf("Enter position of first bit to swap (p): ");
    scanf("%d", &p);
    printf("Enter position of second bit to swap (q): ");
    scanf("%d", &q);

    // First, we check if the bits at position p and q are different.
    // If they are different, then we toggle both bits.
    // To check if they are different, we shift x right by p and q positions and check the least significant bit.
    // To toggle the bits, we use XOR (^) with (1 << p) and (1 << q).
    if (((x >> p) & 1) != ((x >> q) & 1)) {
        x ^= (1 << p) | (1 << q);
    }

    printf("\nValue of %d (%s) after swapping bits at positions %d and %d is: %d (%s)\n", x, decimalToBinaryWithSpace(x), p, q, x, decimalToBinaryWithSpace(x));

    return 0;
}
```

## Output:

```
~ ./swap_bits
Enter an integer (x): 268541
Enter position of first bit to swap (p): 9
Enter position of second bit to swap (q): 15

Value of 268541 ( 0000 0000 0100 0001 1000 1111 1101) after swapping bits at positions 9 and 15 is: 268541 ( 0000 0000 0100 0001 1000 1111 1101)
```

## 34) Detect if Two Integers have Opposite Signs

bool opposite = (x ^ y) < 0;

## Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h> // For bool data type

// Function to convert decimal to binary with spaces for every 4 bits
char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x, y;

    // Input the values
    printf("Enter the first integer (x): ");
    scanf("%d", &x);
    printf("Enter the second integer (y): ");
    scanf("%d", &y);

    // We use the XOR (^) operation on x and y.
    // If the result is negative, it implies that x and y have opposite signs.
    // This is because the sign bit (the leftmost bit) of the two numbers is different.
    bool opposite = (x ^ y) < 0;
    printf("\nDo %d (%s) and %d (%s) have opposite signs? %s\n", x, decimalToBinaryWithSpace(x), y, decimalToBinaryWithSpace(y), opposite ? "Yes" : "No");

    return 0;
}
```

## Output:

```
~ ./detect_opp_sign
Enter the first integer (x): 891
Enter the second integer (y): -692

Do 891 ( 0000 0000 0000 0000 0011 0111 1011) and -692 ( 1111 1111 1111 1111 1101 0100 1100) have opposite signs? Yes
```

## 35) Get Absolute Value without Branching

int mask = x >> (sizeof(int) * 8 - 1);
int abs_val = (x + mask) ^ mask;

## Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>

// Function to convert decimal to binary with spaces for every 4 bits
char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x;

    // Input the value
    printf("Enter an integer (x): ");
    scanf("%d", &x);

    // The operation (x >> (sizeof(int) * 8 - 1)) will produce -1 if x is negative and 0 if x is positive.
    // If x is negative, adding -1 will decrement x, and XOR operation with -1 will flip all bits (equivalent to two's complement).
    // If x is positive, adding 0 won't change x, and XOR operation with 0 won't change x either.
    int mask = x >> (sizeof(int) * 8 - 1);
    int abs_val = (x + mask) ^ mask;
    printf("\nAbsolute value of %d (%s) is: %d (%s)\n", x, decimalToBinaryWithSpace(x), abs_val, decimalToBinaryWithSpace(abs_val));

    return 0;
}
```

## Output:

```
~ ./absolute_value_without_branching
Enter an integer (x): -9854

Absolute value of -9854 ( 1111 1111 1111 1101 1001 1000 0010) is: 9854 ( 0000 0000 0000 0010 0110 0111 1110)
```

## 36) Check if a number is even or odd:

```
if (x & 1) {
    // odd
} else {
    // even
}
```

## Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>

// Function to convert decimal to binary with spaces for every 4 bits
char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x;

    // Input the value
    printf("Enter an integer (x): ");
    scanf("%d", &x);

    // The operation (x & 1) checks the least significant bit of x.
    // If the least significant bit is set (1), then the number is odd.
    // If the least significant bit is not set (0), then the number is even.
    if (x & 1) {
        printf("\n%d (%s) is odd.\n", x, decimalToBinaryWithSpace(x));
    } else {
        printf("\n%d (%s) is even.\n", x, decimalToBinaryWithSpace(x));
    }

    return 0;
}
```

## Output:

```
~ ./even_or_odd
Enter an integer (x): 2513

2513 ( 0000 0000 0000 0000 1001 1101 0001) is odd.
~ ./even_or_odd
Enter an integer (x): 2514

2514 ( 0000 0000 0000 0000 1001 1101 0010) is even.
```

## 37) Find Minimum or Maximum without branching:

int y;
int minimum = y ^ ((x ^ y) & -(x < y));
int maximum = x ^ ((x ^ y) & -(x < y));

## Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>

// Function to convert decimal to binary with spaces for every 4 bits
char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x, y;

    // Input the values
    printf("Enter the first integer (x): ");
    scanf("%d", &x);
    printf("Enter the second integer (y): ");
    scanf("%d", &y);

    // The expression (x < y) returns 1 if true, 0 if false.
    // Taking the negative of this result gives -1 if x < y, 0 otherwise.
    // The operation (x ^ y) gives a value that has 1s in the positions where x and y are different.
    // ANDing this with the negative of the comparison result effectively chooses y if x < y, x otherwise.
    // Finally, XORing with y or x gives the minimum or maximum value respectively.
    int minimum = y ^ ((x ^ y) & -(x < y));
    int maximum = x ^ ((x ^ y) & -(x < y));

    printf("\nBetween %d (%s) and %d (%s):\n", x, decimalToBinaryWithSpace(x), y, decimalToBinaryWithSpace(y));
    printf("Minimum is: %d (%s)\n", minimum, decimalToBinaryWithSpace(minimum));
    printf("Maximum is: %d (%s)\n", maximum, decimalToBinaryWithSpace(maximum));

    return 0;
}
```

## Output:

```
~ ./min_max_no_branch
Enter the first integer (x): 4458
Enter the second integer (y): 2360

Between 4458 ( 0000 0000 0000 0001 0001 0110 1010) and 2360 ( 0000 0000 0000 0000 1001 0011 1000):
Minimum is: 2360 ( 0000 0000 0000 0000 1001 0011 1000)
Maximum is: 4458 ( 0000 0000 0000 0001 0001 0110 1010)
```

## 38) Swap two numbers without a temporary variable:

x = x ^ y;
y = x ^ y;
x = x ^ y;

## Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>

// Function to convert decimal to binary with spaces for every 4 bits
char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x, y;

    // Input the values
    printf("Enter the first integer (x): ");
    scanf("%d", &x);
    printf("Enter the second integer (y): ");
    scanf("%d", &y);

    printf("\nBefore swapping: x = %d (%s), y = %d (%s)\n", x, decimalToBinaryWithSpace(x), y, decimalToBinaryWithSpace(y));

    // The XOR operation can be used to swap two numbers without a temporary variable.
    // The idea is that XOR of a number with itself is 0, and XOR of a number with 0 is the number itself.
    // So, by performing XOR operations sequentially, we can effectively swap the values of x and y.
    x = x ^ y;
    y = x ^ y;  // At this point, y has the original value of x
    x = x ^ y;  // At this point, x has the original value of y

    printf("\nAfter swapping: x = %d (%s), y = %d (%s)\n", x, decimalToBinaryWithSpace(x), y, decimalToBinaryWithSpace(y));

    return 0;
}
```

## Output:

```
~ ./swap_no_remp_var
Enter the first integer (x): 4581
Enter the second integer (y):
-895

Before swapping: x = 4581 ( 0000 0000 0000 0001 0001 1110 0101), y = -895 ( 1111 1111 1111 1111 1100 1000 0001)

After swapping: x = -895 ( 1111 1111 1111 1111 1100 1000 0001), y = 4581 ( 0000 0000 0000 0001 0001 1110 0101)
```

## 39) Compute modulus division by a power of 2:

int divisor = 4;  // (which is 2^2)
int result = x & (divisor - 1);

## Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>

// Function to convert decimal to binary with spaces for every 4 bits
char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x;

    // Input the value of x
    printf("Enter an integer (x): ");
    scanf("%d", &x);

    // When we want to compute the modulus of a number by a power of 2 (like 2, 4, 8, 16, ...),
    // we can use a quick trick with bitwise operations.
    // This method involves ANDing the number with (power of 2 - 1).
    // The result of this operation gives the remainder when the number is divided by the power of 2.
    int divisor = 4;  // (which is 2^2)
    int result = x & (divisor - 1);

    printf("\n%d (%s) modulus %d is: %d (%s)\n", x, decimalToBinaryWithSpace(x), divisor, result, decimalToBinaryWithSpace(result));

    return 0;
}
```

## Output:

```
~ ./modulus_power_2
Enter an integer (x): -9568

-9568 ( 1111 1111 1111 1101 1010 1010 0000) modulus 4 is: 0 ( 0000 0000 0000 0000 0000 0000 0000)
~ ./modulus_power_2
Enter an integer (x): 2514

2514 ( 0000 0000 0000 0000 1001 1101 0010) modulus 4 is: 2 ( 0000 0000 0000 0000 0000 0000 0010)
~ ./modulus_power_2
Enter an integer (x): 36923

36923 ( 0000 0000 0000 1001 0000 0011 1011) modulus 4 is: 3 ( 0000 0000 0000 0000 0000 0000 0011)
```

## 40) Find average of two numbers without overflow:

int average = (x & y) + ((x ^ y) >> 1);

## Program for the above:

```c
#include <stdio.h>
#include <stdlib.h>

// Function to convert decimal to binary with spaces for every 4 bits
char* decimalToBinaryWithSpace(int num) {
    char* binary = (char*)malloc(36);  // 32 bits + 3 spaces + null terminator for an int
    binary[35] = '\0';
    int index = 34;

    for(int i = 0; i < 32; i++) {
        if (i > 0 && i % 4 == 0) {
            binary[index--] = ' '; // Add space every 4 bits
        }
        if((num & 1) == 1) {
            binary[index] = '1';
        } else {
            binary[index] = '0';
        }
        num >>= 1;
        index--;
    }
    return binary;
}

int main() {
    int x, y;

    // Input the values of x and y
    printf("Enter the first integer (x): ");
    scanf("%d", &x);
    printf("Enter the second integer (y): ");
    scanf("%d", &y);

    // Calculating the average directly as (x+y)/2 can cause overflow if x and y are both large.
    // This method avoids overflow by focusing on common bits (x & y) and bits where they differ ((x ^ y) >> 1).
    // The result of these operations is then added to get the average.
    int average = (x & y) + ((x ^ y) >> 1);

    printf("\nThe average of %d (%s) and %d (%s) is: %d (%s)\n", x, decimalToBinaryWithSpace(x), y, decimalToBinaryWithSpace(y), average, decimalToBinaryWithSpace(average));

    return 0;
}
```

## Output:

```
~ ./averge_without_overflow
Enter the first integer (x): 45812
Enter the second integer (y): 33692

The average of 45812 ( 0000 0000 0000 1011 0010 1111 0100) and 33692 ( 0000 0000 0000 1000 0011 1001 1100) is: 39752 ( 0000 0000 0000 1001 1011 0100 1000)
```

**41) Check if all bits in a number are 0 (number is zero):**

bool isZero = !x;

**42) Check if only a single bit is set in a number:**

bool isSingleBitSet = x && !(x & (x-1));

**Program for the above:**

```c
#include <stdio.h>
#include <stdbool.h>

int main() {
    int x;

    // Input the value of x
    printf("Enter an integer (x): ");
    scanf("%d", &x);

    // If x is 0, then !x will return 1 (true), otherwise it will return 0 (false).
    bool isZero = !x;
    printf("\nIs %d zero? %s\n", x, isZero ? "Yes" : "No");

    // This technique works by checking if x is non-zero (to handle the special case when x = 0)
    // and then determining if x has exactly one bit set.
    // The expression (x & (x-1)) will unset the rightmost set bit in x.
    // If the result is 0, then x had only one bit set.
    bool isSingleBitSet = x && !(x & (x-1));
    printf("Does %d have only a single bit set? %s\n", x, isSingleBitSet ? "Yes" : "No");

    return 0;
}
```

**Output:**

```
~ ./numberisZero_check_single_bit_set
Enter an integer (x): 0

Is 0 zero? Yes
Does 0 have only a single bit set? No
~ ./numberisZero_check_single_bit_set
Enter an integer (x): 5890

Is 5890 zero? No
Does 5890 have only a single bit set? No
~ ./numberisZero_check_single_bit_set
Enter an integer (x): 1

Is 1 zero? No
Does 1 have only a single bit set? Yes
```

## 43) Compute the next higher power of 2:

x--;
x |= x >> 1;
x |= x >> 2;
x |= x >> 4;
x |= x >> 8;
x |= x >> 16;
x++;

## Program for the above:

```c
#include <stdio.h>

// Function to compute the next higher power of 2 for a given number
unsigned int nextHigherPowerOf2(unsigned int x) {
    // Step 1: Subtract 1 from x. This ensures that if 'x' is already a power of 2, we get the next power of 2.
    x--;

    // Step 2: OR x with its right shift by 1. This starts the process of setting bits.
    x |= x >> 1;

    // Step 3: Continue this process, doubling the shift amount with each step.
    x |= x >> 2;
    x |= x >> 4;
    x |= x >> 8;

    // Step 4: For 32-bit numbers, we stop at a shift of 16.
    x |= x >> 16;

    // Step 5: Finally, add 1 to the result to get the next power of 2.
    x++;

    return x;
}

int main() {
    unsigned int num;

    // Prompt the user to enter a number
    printf("Enter a number: ");
    scanf("%u", &num);

    // Compute the next higher power of 2 and display the result
    unsigned int result = nextHigherPowerOf2(num);
    printf("Next higher power of 2 for %u is: %u\n", num, result);

    return 0;
}
```

## Output:

```
~ ./next_higher_power_2
Enter a number: 52
Next higher power of 2 for 52 is: 64
~ ./next_higher_power_2
Enter a number: 129
Next higher power of 2 for 129 is: 256
~ ./next_higher_power_2
Enter a number: 4025
Next higher power of 2 for 4025 is: 4096
```

## 44) Setting Bits in a Range:

For setting bits from position p to q:

unsigned int setRange(unsigned int x, int p, int q) {
unsigned int mask = ((1 << (q-p+1)) - 1) << p;
return x | mask;
}

## Program for the above:

```c
#include <stdio.h>

// Function to set bits from position p to q in number x
unsigned int setRange(unsigned int x, int p, int q) {
    unsigned int mask = ((1 << (q-p+1)) - 1) << p;
    return x | mask;
}

// Function to print a number in binary format with spaces for every 4 bits
void printBinary(unsigned int x) {
    for (int i = 31; i >= 0; i--) {
        printf("%d", (x >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }
    printf("\n");
}

int main() {
    unsigned int num;
    int p, q;

    // Prompt the user to enter a number
    printf("Enter a number: ");
    scanf("%u", &num);
    printf("Original number: %u\n", num);
    printf("Original number in binary: ");
    printBinary(num);

    // Prompt the user to enter positions p and q
    printf("Enter the starting position (p): ");
    scanf("%d", &p);
    printf("Enter the ending position (q): ");
    scanf("%d", &q);

    // Set the bits in the specified range
    unsigned int result = setRange(num, p, q);

    // Display the result in decimal and binary formats
    printf("Number after setting bits from position %d to %d: %u\n", p, q, result);
    printf("Modified number in binary: ");
    printBinary(result);

    return 0;
}
```

## Output:

```
~ ./set_bits_inRange
Enter a number: 2651
Original number: 2651
Original number in binary: 0000 0000 0000 0000 0000 1010 0101 1011
Enter the starting position (p): 7
Enter the ending position (q): 26
Number after setting bits from position 7 to 26: 134217691
Modified number in binary: 0000 0111 1111 1111 1111 1111 1101 1011
```

**45) Testing Multiple Bits:**

To check if specific bits are set/cleared:

```
bool areBitsSet(unsigned int x, unsigned int mask) {
return (x & mask) == mask;
}
```

**Output:**

```
/~ ./test_multiple_bits
Enter a number: 5941
Number in decimal: 5941
Number in binary: 0000 0000 0000 0000 0001 0111 0011 0101
Enter the mask: 16
Mask in decimal: 16
Mask in binary: 0000 0000 0000 0000 0000 0000 0001 0000
The specified bits are set in the number.
```

**Program for the above:**

```c
#include <stdio.h>
#include <stdbool.h>

// Function to check if specific bits are set/cleared
bool areBitsSet(unsigned int x, unsigned int mask) {
    return (x & mask) == mask;
}

// Function to print a number in binary format with spaces for every 4 bits
void printBinary(unsigned int x) {
    for (int i = 31; i >= 0; i--) {
        printf("%d", (x >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }
    printf("\n");
}

int main() {
    unsigned int num, mask;

    // Prompt the user to enter a number
    printf("Enter a number: ");
    scanf("%u", &num);
    printf("Number in decimal: %u\n", num);
    printf("Number in binary: ");
    printBinary(num);

    // Prompt the user to enter the mask
    printf("Enter the mask: ");
    scanf("%u", &mask);
    printf("Mask in decimal: %u\n", mask);
    printf("Mask in binary: ");
    printBinary(mask);

    // Check if specific bits are set
    bool result = areBitsSet(num, mask);

    // Display the result
    if(result) {
        printf("The specified bits are set in the number.\n");
    } else {
        printf("The specified bits are not all set in the number.\n");
    }

    return 0;
}

```

**46) Conditional Bit Set/Clear:**

Setting or clearing a bit based on a condition (often used in peripheral configurations):

```
if (condition) {
x |= (1 << BIT_POS); // Set bit if condition is true
} else {
x &= ~(1 << BIT_POS); // Clear bit if condition is false
}
```

**Output:**

```
~ ./conditional_bit_set_clear
Enter a number: 5623
Number in decimal: 5623
Number in binary: 0000 0000 0000 0000 0001 0101 1111 0111
Enter the bit position (0-31): 3
Enter the condition (1 for true, 0 for false): 1
Modified number in decimal: 5631
Modified number in binary: 0000 0000 0000 0000 0001 0101 1111 1111
```

**Program for the above:**

```c
#include <stdio.h>
#include <stdbool.h>  // Required for bool data type

// Function to set or clear a bit based on a condition
void conditionalBitSetClear(unsigned int *x, bool condition, int BIT_POS) {
    if (condition) {
        *x |= (1 << BIT_POS); // Set bit if condition is true
    } else {
        *x &= ~(1 << BIT_POS); // Clear bit if condition is false
    }
}

// Function to print a number in binary format with spaces for every 4 bits
void printBinary(unsigned int x) {
    for (int i = 31; i >= 0; i--) {
        printf("%d", (x >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }
    printf("\n");
}

int main() {
    unsigned int num;
    int BIT_POS;
    bool condition;

    // Prompt the user to enter a number
    printf("Enter a number: ");
    scanf("%u", &num);
    printf("Number in decimal: %u\n", num);
    printf("Number in binary: ");
    printBinary(num);

    // Prompt the user to enter the bit position
    printf("Enter the bit position (0-31): ");
    scanf("%d", &BIT_POS);

    // Prompt the user to specify the condition (1 for true, 0 for false)
    printf("Enter the condition (1 for true, 0 for false): ");
    int tempCondition;
    scanf("%d", &tempCondition);
    condition = (tempCondition == 1);

    // Perform conditional bit set/clear
    conditionalBitSetClear(&num, condition, BIT_POS);

    printf("Modified number in decimal: %u\n", num);
    printf("Modified number in binary: ");
    printBinary(num);

    return 0;
}
```

## 47) Masking Interrupts:

To disable certain interrupts by setting respective bits in interrupt mask registers:

#define INTERRUPT_MASK_ADDR 0x40021050
(volatile unsigned int)INTERRUPT_MASK_ADDR |= (1 << INTERRUPT_BIT); // Mask interrupt

## Program for the above:

```c
#include <stdio.h>

// Use a local variable to simulate the interrupt mask register
volatile unsigned int mock_INTERRUPT_MASK_ADDR = 0;

// For embedded devices, define the address of the interrupt mask register as a pointer to a volatile unsigned int

// Function to print a number in binary format with spaces for every 4 bits
void printBinary(unsigned int x) {
    for (int i = 31; i >= 0; i--) {
        printf("%d", (x >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }
    printf("\n");
}

int main() {
    int INTERRUPT_BIT;

    // Display the current value of the mock interrupt mask register
    printf("Initial mock interrupt mask register value in binary: ");
    printBinary(mock_INTERRUPT_MASK_ADDR);

    // Prompt the user to specify which interrupt bit to mask (disable)
    printf("Enter the interrupt bit to mask (0-31): ");
    scanf("%d", &INTERRUPT_BIT);

    // Mask the specified interrupt
    mock_INTERRUPT_MASK_ADDR |= (1 << INTERRUPT_BIT);

    // Display the updated value of the mock interrupt mask register
    printf("Updated mock interrupt mask register value in binary: ");
    printBinary(mock_INTERRUPT_MASK_ADDR);

    return 0;
}
```

## Output:

```
~ ./masking_interrupt
Initial mock interrupt mask register value in binary: 0000 0000 0000 0000 0000 0000 0000 0000
Enter the interrupt bit to mask (0-31): 25
Updated mock interrupt mask register value in binary: 0000 0010 0000 0000 0000 0000 0000 0000
```

**48) Endianness Swap:**

In situations where data endianness matters (e.g., communication between different systems):

```
unsigned int swapEndianness(unsigned int x) {
return ((x >> 24) & 0xFF) |
((x << 8) & 0xFF0000) |
((x >> 8) & 0xFF00) |
((x << 24) & 0xFF000000);
}
```

**Output:**

```
~ ./endianness_swap
Enter an integer number: 59813
Original number in binary:       0000 0000 0000 0000 1110 1001 1010 0101
Number after endianness swap: 1010 0101 1110 1001 0000 0000 0000 0000
Original number in decimal: 59813
Number after endianness swap in decimal: 2783510528
```

**Program for the above:**

```c
#include <stdio.h>

// Function to perform endianness swap
unsigned int swapEndianness(unsigned int x) {
    return ((x >> 24) & 0xFF) |
           ((x << 8) & 0xFF0000) |
           ((x >> 8) & 0xFF00) |
           ((x << 24) & 0xFF000000);
}

// Function to print a number in binary format with spaces for every 4 bits
void printBinary(unsigned int x) {
    for (int i = 31; i >= 0; i--) {
        printf("%d", (x >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }
    printf("\n");
}

int main() {
    unsigned int num, swappedNum;

    // Prompt the user for an integer input
    printf("Enter an integer number: ");
    scanf("%u", &num);

    // Display the original number in binary
    printf("Original number in binary:       ");
    printBinary(num);

    // Perform endianness swap
    swappedNum = swapEndianness(num);

    // Display the swapped number in binary
    printf("Number after endianness swap: ");
    printBinary(swappedNum);

    // Print both numbers in decimal format
    printf("Original number in decimal: %u\n", num);
    printf("Number after endianness swap in decimal: %u\n", swappedNum);

    return 0;
}
```

**49) Bounded Increment:**

Incrementing a bounded counter (common in timers and PWM modules):

x = (x + 1) & BOUND_MASK;

**Program for the above:**

```c
#include <stdio.h>

/*
 * Bounded Increment:
 * In many embedded systems, especially those with timers and PWM (Pulse Width Modulation) modules,
 * counters need to increment within a specific range. This is often done to avoid overflow or to cycle
 * through a set of values.
 *
 * In this example, we simulate a bounded increment operation using bitwise AND with a mask.
 * The `BOUND_MASK` defines the range or boundary of the counter. By adding 1 to the counter and then
 * performing a bitwise AND with the `BOUND_MASK`, we ensure the counter wraps around upon reaching the
 * boundary.
 */

// Function to perform bounded increment
unsigned int boundedIncrement(unsigned int x, unsigned int BOUND_MASK) {
    return (x + 1) & BOUND_MASK;
}

// Function to print a number in binary format with spaces for every 4 bits
void printBinary(unsigned int x) {
    for (int i = 31; i >= 0; i--) {
        printf("%d", (x >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }
    printf("\n");
}

int main() {
    unsigned int num, incrementedNum, BOUND_MASK;

    // Prompt the user for an integer input
    printf("Enter an integer number: ");
    scanf("%u", &num);

    printf("Enter the BOUND_MASK in hexadecimal (e.g., 0xFF for 8-bit boundary): ");
    scanf("%x", &BOUND_MASK);

    // Display the original number in binary
    printf("Original number in binary:     ");
    printBinary(num);

    // Perform bounded increment
    incrementedNum = boundedIncrement(num, BOUND_MASK);

    // Display the incremented number in binary
    printf("Number after bounded increment: ");
    printBinary(incrementedNum);

    // Print both numbers in decimal format
    printf("Original number in decimal: %u\n", num);
    printf("Number after bounded increment in decimal: %u\n", incrementedNum);

    return 0;
}
```

**Output:**

```
~ ./bounded_increment
Enter an integer number: 2613
Enter the BOUND_MASK in hexadecimal (e.g., 0xFF for 8-bit boundary): 0x23
Original number in binary:     0000 0000 0000 0000 0000 1010 0011 0101
Number after bounded increment: 0000 0000 0000 0000 0000 0000 0010 0010
Original number in decimal: 2613
Number after bounded increment in decimal: 34
~ ./bounded_increment
Enter an integer number: 2613
Enter the BOUND_MASK in hexadecimal (e.g., 0xFF for 8-bit boundary): 23
Original number in binary:     0000 0000 0000 0000 0000 1010 0011 0101
Number after bounded increment: 0000 0000 0000 0000 0000 0000 0010 0010
Original number in decimal: 2613
Number after bounded increment in decimal: 34
```

**50) Hardware Loop Unrolling for Bitwise Operations:**

For operations on arrays of bits, unroll loops to make the code more predictable and often faster:

```
for (int i = 0; i < ARRAY_SIZE; i+=4) {
arr[i] |= MASK;
arr[i+1] |= MASK;
arr[i+2] |= MASK;
arr[i+3] |= MASK;

}
```

**Output:**

```
~ ./hardware_unroll_bitwise
Modified array:
Element 0: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
Element 1: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
Element 2: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
Element 3: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
Element 4: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
Element 5: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
Element 6: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
Element 7: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
Element 8: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
Element 9: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
Element 10: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
Element 11: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
Element 12: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
Element 13: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
Element 14: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
Element 15: Decimal: 15, Binary: 0000 0000 0000 0000 0000 0000 0000 1111
```

## Program for the above:

```c
#include <stdio.h>

/*
 * Hardware Loop Unrolling for Bitwise Operations:
 * Loop unrolling is a common optimization technique used in programming, especially in embedded systems
 * where predictable execution and performance is crucial. By unrolling loops, the overhead of the loop
 * control is reduced, and in many cases, the code can be executed faster.
 *
 * This technique is beneficial in scenarios where the operations inside the loop are simple (like bitwise
 * operations) and can be executed in parallel by the hardware.
 *
 * In this example, we demonstrate loop unrolling for an operation on an array of integers. Instead of
 * performing a bitwise OR operation with a MASK on each array element one by one, we perform the operation
 * on four array elements at once, reducing the total number of loop iterations.
 */

#define ARRAY_SIZE 16
#define MASK 0x0F  // Example mask

// Function to perform bitwise OR operation on array elements using loop unrolling
void applyMaskUnrolled(unsigned int arr[], int size) {
    for (int i = 0; i < size; i += 4) {
        arr[i] |= MASK;
        arr[i + 1] |= MASK;
        arr[i + 2] |= MASK;
        arr[i + 3] |= MASK;
    }
}

int main() {
    unsigned int arr[ARRAY_SIZE];

    // Initialize the array with sample values
    for (int i = 0; i < ARRAY_SIZE; i++) {
        arr[i] = i;
    }

    // Apply the mask using loop unrolling
    applyMaskUnrolled(arr, ARRAY_SIZE);

    // Print the modified array
    printf("Modified array:\n");
    for (int i = 0; i < ARRAY_SIZE; i++) {
        printf("Element %d: Decimal: %u, Binary: ", i, arr[i]);
        for (int j = 31; j >= 0; j--) {
            printf("%d", (arr[i] >> j) & 1);
            if (j % 4 == 0) {
                printf(" ");
            }
        }
        printf("\n");
    }

    return 0;
}
```

## 51) Power-of-Two Boundary Alignment:

Aligning addresses/data on power-of-two boundaries is often crucial for memory access and DMA transfers:

x = (x + BOUNDARY - 1) & ~(BOUNDARY - 1);

**Program for the above:**

```c
#include <stdio.h>

/*
 * Power-of-Two Boundary Alignment:
 * In embedded systems, aligning data or addresses on power-of-two boundaries is often essential.
 * This could be due to requirements for efficient memory access, DMA transfers, or specific hardware
 * constraints.
 *
 * When data or an address needs to be aligned to a specific boundary (like 4, 8, 16, etc.), the following
 * technique can be used. The idea is to add the (boundary - 1) to the value, and then mask off the lower
 * bits, ensuring that the result is aligned to the desired boundary.
 *
 * The given formula:
 *     x = (x + BOUNDARY - 1) & ~(BOUNDARY - 1)
 * ensures that the number 'x' is rounded up to the nearest multiple of BOUNDARY.
 */

#define BOUNDARY 8  // Example boundary

// Function to align a value to a power-of-two boundary
unsigned int alignToBoundary(unsigned int x) {
    return (x + BOUNDARY - 1) & ~(BOUNDARY - 1);
}

int main() {
    unsigned int value = 13;  // Sample value

    // Align the value to the boundary
    unsigned int alignedValue = alignToBoundary(value);

    // Print the results
    printf("Original Value: Decimal: %u, Binary: ", value);
    for (int j = 31; j >= 0; j--) {
        printf("%d", (value >> j) & 1);
        if (j % 4 == 0) {
            printf(" ");
        }
    }
    printf("\n");

    printf("Aligned Value: Decimal: %u, Binary: ", alignedValue);
    for (int j = 31; j >= 0; j--) {
        printf("%d", (alignedValue >> j) & 1);
        if (j % 4 == 0) {
            printf(" ");
        }
    }
    printf("\n");

    return 0;
}
```

**Output:**

```
~ ./poweOf_2_boundary
Original Value: Decimal: 13, Binary: 0000 0000 0000 0000 0000 0000 0000 1101
Aligned Value: Decimal: 16, Binary: 0000 0000 0000 0000 0000 0000 0001 0000
```

**52) Bit Fields Manipulation in Structs:**

```
struct ControlRegister {
unsigned int mode: 3;
unsigned int enable: 1;
unsigned int flag: 1;
unsigned int reserved: 27;
};
```

**Output:**

```
~ ./struct_bit_manipulation
Control Register Bit Fields:
1. Mode (3 bits)
2. Enable (1 bit)
3. Flag (1 bit)
4. Reserved (27 bits)

Enter Mode (0-7): 1
Enter Enable (0-1): 0
Enter Flag (0-1): 1

Updated Control Register:
Mode: 1
Enable: 0
Flag: 1
Reserved: 0
```

## Program for the above:

```c
#include <stdio.h>

/*
 * Bit Fields Manipulation in Structs:
 * Embedded systems often interface with hardware where registers have specific bits dedicated
 * to certain functions. Bit fields in C structures are an efficient way to map software constructs
 * onto such hardware registers.
 *
 * In this example, a ControlRegister structure maps to a 32-bit register with various bit fields.
 */

struct ControlRegister {
    unsigned int mode: 3;
    unsigned int enable: 1;
    unsigned int flag: 1;
    unsigned int reserved: 27;
};

int main() {
    struct ControlRegister reg = {0};
    unsigned int tempMode, tempEnable, tempFlag;

    printf("Control Register Bit Fields:\n");
    printf("1. Mode (3 bits)\n");
    printf("2. Enable (1 bit)\n");
    printf("3. Flag (1 bit)\n");
    printf("4. Reserved (27 bits)\n\n");

    printf("Enter Mode (0-7): ");
    scanf("%u", &tempMode);
    reg.mode = tempMode;

    printf("Enter Enable (0-1): ");
    scanf("%u", &tempEnable);
    reg.enable = tempEnable;

    printf("Enter Flag (0-1): ");
    scanf("%u", &tempFlag);
    reg.flag = tempFlag;

    // Reserved bits are typically not set by the user as they are reserved for future use
    // or specific internal hardware functions. Hence, we won't take input for them.

    printf("\nUpdated Control Register:\n");
    printf("Mode: %u\n", reg.mode);
    printf("Enable: %u\n", reg.enable);
    printf("Flag: %u\n", reg.flag);
    printf("Reserved: %u\n", reg.reserved);  // This will typically be 0 unless set elsewhere

    return 0;
}
```

**53) Parity Check:**

bool hasEvenParity(unsigned int x) {
x ^= x >> 16;
x ^= x >> 8;
x ^= x >> 4;
x ^= x >> 2;
x ^= x >> 1;
return !(x & 1);
}

**Program for the above:**

```c
#include <stdio.h>
#include <stdbool.h>

/*
 * Parity Check:
 * In digital communications, parity check is often used to detect single bit errors during data transmission.
 * The function 'hasEvenParity' checks if the given number has an even number of set bits (even parity).
 *
 * The function uses bitwise operations to successively reduce the number of set bits in 'x'
 * and checks the least significant bit to determine the overall parity.
 */

bool hasEvenParity(unsigned int x) {
    x ^= x >> 16;
    x ^= x >> 8;
    x ^= x >> 4;
    x ^= x >> 2;
    x ^= x >> 1;
    return !(x & 1);
}

void printBinaryWithSpaces(unsigned int num) {
    for (int i = 31; i >= 0; i--) {
        printf("%u", (num >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }
    printf("\n");
}

int main() {
    unsigned int num;

    printf("Enter a number: ");
    scanf("%u", &num);

    printf("Binary representation: ");
    printBinaryWithSpaces(num);

    if(hasEvenParity(num)) {
        printf("The number %u has even parity.\n", num);
    } else {
        printf("The number %u has odd parity.\n", num);
    }

    return 0;
}

```

**Output:**

```
~ ./parity_check
Enter a number: 5124
Binary representation: 0000 0000 0000 0000 0001 0100 0000 0100
The number 5124 has odd parity.
```

## 54) Counting Bits in a Byte:

const unsigned char BitsSetTable256[256] = {...};
unsigned int countSetBits(unsigned int x) {
return BitsSetTable256[x & 0xFF] +
BitsSetTable256[(x >> 8) & 0xFF] +
BitsSetTable256[(x >> 16) & 0xFF] +
BitsSetTable256[x >> 24];
}

## Program for the above:

```c
#include <stdio.h>

/*
 * Counting Bits in a Byte:
 * To speed up counting set bits in a number, one common technique is to use a lookup table.
 * 'BitsSetTable256' is a predefined table where each entry represents the number of set bits for that index.
 * The function 'countSetBits' uses this table to quickly count the set bits in the given number 'x'.
 */

// Assuming a table with precalculated values of set bits in each byte.
const unsigned char BitsSetTable256[256] = {
    // ... (values from 0 to 8 depending on the number of bits set in the corresponding index)
};

unsigned int countSetBits(unsigned int x) {
    return BitsSetTable256[x & 0xFF] +
           BitsSetTable256[(x >> 8) & 0xFF] +
           BitsSetTable256[(x >> 16) & 0xFF] +
           BitsSetTable256[x >> 24];
}

void printBinaryWithSpaces(unsigned int num) {
    for (int i = 31; i >= 0; i--) {
        printf("%u", (num >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }
    printf("\n");
}

int main() {
    unsigned int num;

    printf("Enter a number: ");
    scanf("%u", &num);

    printf("Binary representation: ");
    printBinaryWithSpaces(num);

    printf("The number %u has %u set bits.\n", num, countSetBits(num));

    return 0;
}
```

**Output:**

```
~ ./count_bits_in_bytes
Enter a number: 487
Binary representation: 0000 0000 0000 0000 0000 0001 1110 0111
The number 487 has 0 set bits.
```

**55) Bit Reversal:**

```
unsigned int reverseBits(unsigned int x) {
x = ((x & 0x55555555) << 1) | ((x & 0xAAAAAAAA) >> 1);
x = ((x & 0x33333333) << 2) | ((x & 0xCCCCCCCC) >> 2);
x = ((x & 0x0F0F0F0F) << 4) | ((x & 0xF0F0F0F0) >> 4);
x = ((x & 0x00FF00FF) << 8) | ((x & 0xFF00FF00) >> 8);
return (x << 16) | (x >> 16);
}
```

**Output:**

```
~ ./bit_reversal
Enter a number: 84752
Original number in binary: 0000 0000 0000 0001 0100 1011 0001 0000
Reversed number in binary: 0000 1000 1101 0010 1000 0000 0000 0000
```

**Program for the above:**

```c
#include <stdio.h>

/*
 * Bit Reversal:
 * The function 'reverseBits' uses a series of operations to reverse the bits of a given number.
 * It systematically swaps bit positions in pairs, nibbles, bytes, and finally half-words.
 */

unsigned int reverseBits(unsigned int x) {
    x = ((x & 0x55555555) << 1) | ((x & 0xAAAAAAAA) >> 1);
    x = ((x & 0x33333333) << 2) | ((x & 0xCCCCCCCC) >> 2);
    x = ((x & 0x0F0F0F0F) << 4) | ((x & 0xF0F0F0F0) >> 4);
    x = ((x & 0x00FF00FF) << 8) | ((x & 0xFF00FF00) >> 8);
    return (x << 16) | (x >> 16);
}

void printBinaryWithSpaces(unsigned int num) {
    for (int i = 31; i >= 0; i--) {
        printf("%u", (num >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }
    printf("\n");
}

int main() {
    unsigned int num, reversed;

    printf("Enter a number: ");
    scanf("%u", &num);

    reversed = reverseBits(num);

    printf("Original number in binary: ");
    printBinaryWithSpaces(num);

    printf("Reversed number in binary: ");
    printBinaryWithSpaces(reversed);

    return 0;
}
```

## 56) Low-Level Port Manipulation:

#define PORT_ADDR 0x40021000
#define PIN_MASK (1 << 5)
(volatile unsigned int)PORT_ADDR |= PIN_MASK;

**Program for the above:**

```c
#include <stdio.h>

/*
 * Low-Level Port Manipulation:
 * Direct manipulation of hardware ports is common in embedded systems.
 * In this example, we set a specific pin (pin 5) of a port located at address 0x40021000.
 */

#define PORT_ADDR 0x40021000  // Mock address for demonstration purposes. In real time, use required port pin addr.
#define PIN_MASK (1 << 5)

// Mock memory space for the demonstration
unsigned int mockMemorySpace[0x40022000];

// Mock function for reading and writing to our simulated memory space
unsigned int *simulateMemoryAccess(unsigned int addr) {
    return &mockMemorySpace[addr];
}

int main() {
    // Simulate the direct access to the memory-mapped I/O register
    unsigned int *port = simulateMemoryAccess(PORT_ADDR);
    *port |= PIN_MASK;

    printf("Port value after manipulation: ");
    for (int i = 31; i >= 0; i--) {
        printf("%u", (*port >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }
    printf("\n");

    return 0;
}

```

**Output:**

```
~ ./port_maniup
Port value after manipulation: 0000 0000 0000 0000 0000 0000 0010 0000
```

## 57) Bit Extraction:

unsigned int extractBits(unsigned int x, int p, int q) {
return (x >> p) & ((1 << (q-p+1)) - 1);
}

## Program for the above:

```c
#include <stdio.h>

/*
 * Bit Extraction:
 * In embedded systems, it's common to extract specific bits from registers or data words.
 * This function extracts bits from position p to q (inclusive) from the number x.
 */

unsigned int extractBits(unsigned int x, int p, int q) {
    return (x >> p) & ((1 << (q-p+1)) - 1);
}

int main() {
    unsigned int x;
    int p, q;

    // User input for the number and bit positions
    printf("Enter the number (decimal): ");
    scanf("%u", &x);
    printf("Enter starting position p: ");
    scanf("%d", &p);
    printf("Enter ending position q: ");
    scanf("%d", &q);

    // Extract bits and display results
    unsigned int result = extractBits(x, p, q);
    printf("\nOriginal number (binary): ");
    for (int i = 31; i >= 0; i--) {
        printf("%u", (x >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }
    printf("\nExtracted bits (binary):  ");
    for (int i = (q-p); i >= 0; i--) {
        printf("%u", (result >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }
    printf("\nExtracted bits (decimal): %u\n", result);

    return 0;
}
```

## Output:

```
./bit_extraction
Enter the number (decimal): 1254
Enter starting position p: 2
Enter ending position q: 9

Original number (binary): 0000 0000 0000 0000 0000 0100 1110 0110
Extracted bits (binary):  0011 1001
Extracted bits (decimal): 57
```

**58) Boundary Checks:**

bool isWithinBoundary(unsigned int x, unsigned int boundary) {
return (x & (boundary - 1)) == x;
}

**Program for the above:**

```c
#include <stdio.h>
#include <stdbool.h>

/*
 * Boundary Checks:
 * In embedded systems, ensuring data or addresses stay within specific boundaries
 * (often power-of-two limits) is essential. This function checks if a number x
 * is within the range [0, boundary-1].
 */

bool isWithinBoundary(unsigned int x, unsigned int boundary) {
    return x < boundary;
}

int main() {
    unsigned int x, boundary;

    // User input for the number and the boundary
    printf("Enter the number (decimal): ");
    scanf("%u", &x);
    printf("Enter the boundary (decimal): ");
    scanf("%u", &boundary);

    // Check boundary and display results
    bool withinBoundary = isWithinBoundary(x, boundary);
    printf("\nNumber (decimal): %u\n", x);
    printf("Number (binary):  ");
    for (int i = 31; i >= 0; i--) {
        printf("%u", (x >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }

    // Print the result
    if (withinBoundary) {
        printf("\nThe number %u is within the boundary [0, %u).\n", x, boundary);
    } else {
        printf("\nThe number %u is NOT within the boundary [0, %u).\n", x, boundary);
    }

    return 0;
}

```

**Output:**

```
~ ./boundary_check
Enter the number (decimal): 4457
Enter the boundary (decimal): 2513

Number (decimal): 4457
Number (binary):  0000 0000 0000 0000 0001 0001 0110 1001
The number 4457 is NOT within the boundary [0, 2513).
```

## 59) Bit Rotation:

unsigned int rotateLeft(unsigned int x, int n) {
return (x << n) | (x >> (sizeof(x) * 8 - n));
}

## Program for the above:

```c
#include <stdio.h>

/*
 * Bit Rotation:
 * Rotating bits is a common operation in cryptographic algorithms and digital signal processing.
 * This function rotates the bits of number x to the left by n positions.
 */

unsigned int rotateLeft(unsigned int x, int n) {
    return (x << n) | (x >> (sizeof(x) * 8 - n));
}

int main() {
    unsigned int x;
    int n;

    // User input for the number and the rotation count
    printf("Enter the number (decimal): ");
    scanf("%u", &x);
    printf("Enter the number of positions to rotate left: ");
    scanf("%d", &n);

    // Perform bit rotation and store the result
    unsigned int result = rotateLeft(x, n);

    // Display results
    printf("\nOriginal Number (decimal): %u\n", x);
    printf("Original Number (binary):  ");
    for (int i = 31; i >= 0; i--) {
        printf("%u", (x >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }

    printf("\nRotated Number (decimal): %u\n", result);
    printf("Rotated Number (binary):  ");
    for (int i = 31; i >= 0; i--) {
        printf("%u", (result >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }
    printf("\n");

    return 0;
}
```

**Output:**

```
~ ./bit_rotation
Enter the number (decimal): 587
Enter the number of positions to rotate left: 25

Original Number (decimal): 587
Original Number (binary):  0000 0000 0000 0000 0000 0010 0100 1011
Rotated Number (decimal): 2516582404
Rotated Number (binary):  1001 0110 0000 0000 0000 0000 0000 0100
```

## 60) Integer Average without Overflow:

int average = (x & y) + ((x ^ y) >> 1);

**Output:**

```
~ ./average_without_overflow
Enter the first number: 457
Enter the second number: 2365

Given numbers:
x (decimal): 457
x (binary):  0000 0000 0000 0000 0000 0001 1100 1001
y (decimal): 2365
y (binary):  0000 0000 0000 0000 0000 1001 0011 1101

Average (decimal): 1411
Average (binary):  0000 0000 0000 0000 0000 0101 1000 0011
```

**Program for the above:**

```c
#include <stdio.h>

// Function to compute the average without overflow
int computeAverage(int x, int y) {
    int average = (x & y) + ((x ^ y) >> 1);
    return average;
}

int main() {
    // Taking two integer inputs from the user
    int x, y;
    printf("Enter the first number: ");
    scanf("%d", &x);
    printf("Enter the second number: ");
    scanf("%d", &y);

    // Compute the average
    int avg = computeAverage(x, y);

    // Display the numbers and their average
    printf("\nGiven numbers:\n");
    printf("x (decimal): %d\n", x);
    printf("x (binary):  ");
    for(int i = 31; i >= 0; i--) {
        printf("%d", (x >> i) & 1);
        if (i % 4 == 0) printf(" ");
    }

    printf("\ny (decimal): %d\n", y);
    printf("y (binary):  ");
    for(int i = 31; i >= 0; i--) {
        printf("%d", (y >> i) & 1);
        if (i % 4 == 0) printf(" ");
    }

    printf("\n\nAverage (decimal): %d\n", avg);
    printf("Average (binary):  ");
    for(int i = 31; i >= 0; i--) {
        printf("%d", (avg >> i) & 1);
        if (i % 4 == 0) printf(" ");
    }

    return 0;
}
```

**61) Memory-Mapped Register Manipulation:**

```
#define REGISTER_ADDR 0x40021000
#define BIT_POS 5
*(volatile unsigned int*)REGISTER_ADDR |= (1 << BIT_POS);  // Set bit
*(volatile unsigned int*)REGISTER_ADDR &= ~(1 << BIT_POS); // Clear bit
```

**Output:**

```
~ ./memory_mapped_reg_manipul
After setting the bit:
Value (decimal): 32
Value (binary):  0000 0000 0000 0000 0000 0000 0010 0000

After clearing the bit:
Value (decimal): 0
Value (binary):  0000 0000 0000 0000 0000 0000 0000 0000 ~
```

## Program for the above:

```c
/*
Memory-mapped I/O allows for both memory and I/O devices to use the same address space.
The key advantage is the ability to use normal instruction operands to address I/O
instead of specialized I/O instructions.

Using the `volatile` keyword before the pointer declaration informs the compiler that the value
at this address can change at any time without any action being taken by the code the
compiler finds nearby. This is crucial when working with hardware registers, as their
values can change asynchronously to the flow of the program.

The `|=` operation is a bitwise OR assignment. It sets the specific bit at position
BIT_POS to 1. This is commonly used to enable certain features or settings in hardware
registers.

The `&= ~` operation is a bitwise AND assignment combined with a bitwise NOT operation.
It clears the bit at position BIT_POS, setting it to 0. This is used to disable or reset
certain features in hardware registers.

In real embedded code, you would directly use the REGISTER_ADDR address to access the
hardware register. Here, for simulation purposes, we're using a pointer to a dummy
variable. Remember to replace `&dummyRegister` with `REGISTER_ADDR` in an actual embedded
environment.
*/

#include <stdio.h>

// For the purpose of this example on a standard PC,
// we will use a dummy address and simulate the register.
#define REGISTER_ADDR 0x40021000
unsigned int dummyRegister = 0;  // Simulating the register

// Bit position to manipulate
#define BIT_POS 5

int main() {
    // For real embedded systems, replace `&dummyRegister` with the actual REGISTER_ADDR.
    volatile unsigned int *registerPtr = &dummyRegister;

    // Set the bit at BIT_POS
    *registerPtr |= (1 << BIT_POS);

    // Display after setting the bit
    printf("After setting the bit:\n");
    printf("Value (decimal): %u\n", *registerPtr);
    printf("Value (binary):  ");
    for (int i = 31; i >= 0; i--) {
        printf("%d", (*registerPtr >> i) & 1);
        if (i % 4 == 0) printf(" ");
    }
    printf("\n");

    // Clear the bit at BIT_POS
    *registerPtr &= ~(1 << BIT_POS);

    // Display after clearing the bit
    printf("\nAfter clearing the bit:\n");
    printf("Value (decimal): %u\n", *registerPtr);
    printf("Value (binary):  ");
    for (int i = 31; i >= 0; i--) {
        printf("%d", (*registerPtr >> i) & 1);
        if (i % 4 == 0) printf(" ");
    }

    return 0;
}
```

## 62) Toggle a GPIO Pin:

#define GPIO_PIN_ADDR 0x40021015
*(volatile unsigned int*)GPIO_PIN_ADDR ^= 1;  // Toggle pin

## Program for the above:

```c
/*
General-Purpose Input/Output (GPIO) pins are versatile pins on a microcontroller or microprocessor.
They can be set to either an input or output mode and can be driven high (usually 3.3V or 5V)
or low (0V). Toggling a GPIO pin means changing its state from high to low or vice versa.

Using the `volatile` keyword before the pointer declaration is crucial when working with
hardware registers. This tells the compiler that the value at this address can change
outside the direct actions of the nearby code, such as by external hardware signals.

The `^=` operation is a bitwise XOR assignment. When XORing with 1, it has the effect
of toggling the least significant bit of the value. This is a common approach to
change the state of a GPIO pin.

In a real embedded environment, the GPIO pin might control an LED, a relay, or any other
digital peripheral. Here, we're simulating this behavior. When porting to an actual
embedded device, replace `&dummyGPIOPin` with `GPIO_PIN_ADDR`.
*/

#include <stdio.h>

// For the purpose of this example on a standard PC,
// we will use a dummy address and simulate the GPIO pin.
#define GPIO_PIN_ADDR 0x40021015
unsigned int dummyGPIOPin = 0;  // Simulating the GPIO pin

int main() {
    // For real embedded systems, replace `&dummyGPIOPin` with the actual GPIO_PIN_ADDR.
    volatile unsigned int *gpioPinPtr = &dummyGPIOPin;

    // Toggle the GPIO pin
    *gpioPinPtr ^= 1;

    // Display after toggling
    printf("After toggling the GPIO pin:\n");
    printf("Value (decimal): %u\n", *gpioPinPtr);
    printf("Value (binary):  ");
    for (int i = 31; i >= 0; i--) {
        printf("%d", (*gpioPinPtr >> i) & 1);
        if (i % 4 == 0) printf(" ");
    }

    return 0;
}
```

## Output:

```
~ ./toggle_gpio
After toggling the GPIO pin:
Value (decimal): 1
Value (binary):  0000 0000 0000 0000 0000 0000 0000 0001
```

**63) Peripheral Enable (common in MCU configurations):**

#define PERIPH_ENABLE_ADDR 0x40021020
#define PERIPH_BIT 3
*(volatile unsigned int*)PERIPH_ENABLE_ADDR |= (1 << PERIPH_BIT); // Enable peripheral

**Program for the above:**

```c
/*
Microcontrollers and microprocessors come with a wide range of peripherals, such as UART,
SPI, I2C, ADC, etc. Before using them, these peripherals often need to be explicitly
enabled. This is typically done by setting a specific bit in a control or configuration
register of the microcontroller.

The `volatile` keyword before the pointer declaration is important. It informs the compiler
that the value at this address can change unexpectedly, perhaps by actions outside of the
current code flow, such as hardware events.

The `|=` operation is a bitwise OR assignment. When OR-ing with a shifted '1' bit, it has
the effect of setting a particular bit to 1, leaving others unchanged. This is a common
approach to enable a peripheral in embedded systems.

In a real-world scenario, enabling a peripheral might activate its clock, initialize its
settings, or make it ready for transactions. In this simulation, we're just changing a
bit in a memory location. When porting to an actual embedded device, replace
`&dummyPeriphRegister` with `PERIPH_ENABLE_ADDR`.
*/

#include <stdio.h>

// For the purpose of this example on a standard PC,
// we will use a dummy address and simulate the peripheral enable register.
#define PERIPH_ENABLE_ADDR 0x40021020
unsigned int dummyPeriphRegister = 0;  // Simulating the register

int main() {
    // For real embedded systems, replace `&dummyPeriphRegister` with the actual PERIPH_ENABLE_ADDR.
    volatile unsigned int *periphRegPtr = &dummyPeriphRegister;

    // Enable the peripheral
    const int PERIPH_BIT = 3;
    *periphRegPtr |= (1 << PERIPH_BIT);

    // Display after enabling the peripheral
    printf("After enabling the peripheral:\n");
    printf("Register Value (decimal): %u\n", *periphRegPtr);
    printf("Register Value (binary):  ");
    for (int i = 31; i >= 0; i--) {
        printf("%d", (*periphRegPtr >> i) & 1);
        if (i % 4 == 0) printf(" ");
    }

    return 0;
}
```

**Output:**

```
~ ./peripheral_enable
After enabling the peripheral:
Register Value (decimal): 8
Register Value (binary):  0000 0000 0000 0000 0000 0000 0000 1000
```

## 64) Changing Bit Fields (common in MCU register configurations):

```
#define CONFIG_REGISTER_ADDR 0x40021030
#define FIELD_MASK 0x70  // Assuming bits 4-6 represent a field
#define FIELD_VALUE 0x50  // New value for bits 4-6
*(volatile unsigned int*)CONFIG_REGISTER_ADDR =
    (*(volatile unsigned int*)CONFIG_REGISTER_ADDR & ~FIELD_MASK) |
FIELD_VALUE;
```

## Program for the above:

```c
/*
Microcontrollers often use specific bits within a register to control or configure particular
features. These subsets of bits are referred to as 'bit fields'. For example, a register might
use bits 4-6 to set the operating mode of a peripheral.

To change just those bits without affecting others, we typically use a combination of bit
masking and bitwise operations:

1. The `~FIELD_MASK` will have all bits set to 1, except for the bits 4-6 which are set to 0.
2. AND-ing this with the current register value will clear the bits 4-6, leaving all other bits
   unchanged.
3. OR-ing the result with the `FIELD_VALUE` will then set the bits 4-6 to the desired value.

The `volatile` keyword informs the compiler that the value can change unexpectedly. This is
important for memory-mapped I/O to ensure the compiler does not optimize out important reads/writes.

In this simulation, we're just changing a bit field in a memory location. When porting to
an actual embedded device, replace `&dummyConfigRegister` with `CONFIG_REGISTER_ADDR`.
*/

#include <stdio.h>

// For the purpose of this example on a standard PC,
// we will use a dummy address and simulate the configuration register.
#define CONFIG_REGISTER_ADDR 0x40021030
unsigned int dummyConfigRegister = 0;  // Simulating the register

int main() {
    // For real embedded systems, replace `&dummyConfigRegister` with the actual CONFIG_REGISTER_ADDR.
    volatile unsigned int *configRegPtr = &dummyConfigRegister;

    // Change the bit field
    const unsigned int FIELD_MASK = 0x70;
    const unsigned int FIELD_VALUE = 0x50;
    *configRegPtr = (*configRegPtr & ~FIELD_MASK) | FIELD_VALUE;

    // Display after changing the bit field
    printf("After changing the bit field:\n");
    printf("Register Value (decimal): %u\n", *configRegPtr);
    printf("Register Value (binary):  ");
    for (int i = 31; i >= 0; i--) {
        printf("%d", (*configRegPtr >> i) & 1);
        if (i % 4 == 0) printf(" ");
    }

    return 0;
}
```

## Output:

```
~ ./changing_bit_fields
After changing the bit field:
Register Value (decimal): 80
Register Value (binary):  0000 0000 0000 0000 0000 0000 0101 0000
```

## 65) Reading a Specific Bit Field:

#define STATUS_REGISTER_ADDR 0x40021040
#define STATUS_MASK 0x0E  // Assuming bits 1-3 are status bits
unsigned int status = (*(volatile unsigned int*)STATUS_REGISTER_ADDR & STATUS_MASK) >> 1;

**Program for the above:**

```c
/*
In embedded systems, registers often contain status or configuration bits that the software needs
to check. To read a specific subset of bits, or a 'bit field' from a register, we employ bit masking
and shifting:

1. Using `STATUS_MASK`, we mask out all other bits except the bits of interest. The AND operation
   (`&`) will clear all bits that are not masked, leaving only the desired bits.
2. Once we have masked the bits, we shift them to the right-most position to interpret or
   represent them as a regular number.

The `volatile` keyword is used to tell the compiler that the value at that memory location can
change unexpectedly, ensuring it doesn't optimize out essential reads.

In our simulation, we are reading a specific bit field from a simulated memory location. On a real
embedded system, you'd replace `&dummyStatusRegister` with `STATUS_REGISTER_ADDR`.
*/

#include <stdio.h>

// For this simulation on a standard PC,
// we use a dummy address and simulate the status register.
#define STATUS_REGISTER_ADDR 0x40021040
unsigned int dummyStatusRegister = 0xF3;  // Simulating the register

int main() {
    // For real embedded systems, replace `&dummyStatusRegister` with the actual STATUS_REGISTER_ADDR.
    volatile unsigned int *statusRegPtr = &dummyStatusRegister;

    // Read the specific bit field
    const unsigned int STATUS_MASK = 0x0E;
    unsigned int status = (*statusRegPtr & STATUS_MASK) >> 1;

    // Display the bit field
    printf("Extracted status bits:\n");
    printf("Status Value (decimal): %u\n", status);
    printf("Status Value (binary):  ");
    for (int i = 2; i >= 0; i--) {  // Only 3 bits (bits 1-3) are our status bits
        printf("%d", (status >> i) & 1);
        if (i % 4 == 0) printf(" ");
    }

    return 0;
}
```

**Output:**

```
~ ./read_bit_field
Extracted status bits:
Status Value (decimal): 1
Status Value (binary):  001
```

Written By: **Yashwanth Naidu Tikkisetty**

Happy learning.

Learn together, Grow together.

Follow me to receive updates regarding Embedded Systems.

Connect with me on LinkedIn: https://www.linkedin.com/in/t-yashwanth-naidu/

T Yashwanth Naidu

GitHub https://github.com/T-Yashwanth-Naidu